Assignment 5. Advanced JavaScript and DOM Manipulation

Project: Essence - perfume shop website

Team name	Essence Team
Member 1	Manyakina Yuliya
Member 2	Bazarbayev Bakdaulet
Group	[WRITE YOUR GROUP HERE]
Deployed website	[PASTE GITHUB PAGES / NETLIFY URL HERE]

1. Objective

The goal of this assignment is to make the Essence perfume shop interactive using JavaScript: DOM manipulation, events, objects, arrays, higher-order functions, sounds and animations. The site has 4 pages: Home, Catalog, Gallery and Contact. HTML, CSS and JavaScript are in separate files (css/style.css and one file per script in the js/ folder).

2. Work distribution

Script	Page	Author
theme.js	all pages	Manyakina Yuliya
nav.js	all pages	Manyakina Yuliya
greeting.js	Home	Manyakina Yuliya
clock.js	Home	Manyakina Yuliya
language.js	Home	Manyakina Yuliya
sound.js	all pages	Bazarbayev Bakdaulet
data.js	Catalog	Bazarbayev Bakdaulet
catalog.js	Catalog	Bazarbayev Bakdaulet
gallery.js	Gallery	Bazarbayev Bakdaulet
rating.js	Gallery	Bazarbayev Bakdaulet
contact.js	Contact	Bazarbayev Bakdaulet

Yuliya made the Home page and the smaller scripts; Bakdaulet made the Catalog, Gallery and Contact pages and the more complex scripts (arrays, fetch, callbacks, sounds). Both members tested the whole site together.

3. Tasks

Task 3.1. Day / Night theme switch (js/theme.js)

Requirement: 1. DOM Manipulation and Styling - Dynamic Style Changes
Author: Manyakina Yuliya

Step-by-step description:
1) Colors are stored in CSS variables. The class "night" on body gives them new values.
2) The button #theme-btn is selected with getElementById.
3) On click, classList.toggle("night") switches the theme and textContent changes the button text.
4) The choice is saved in localStorage, so the theme stays after reload.

Code:

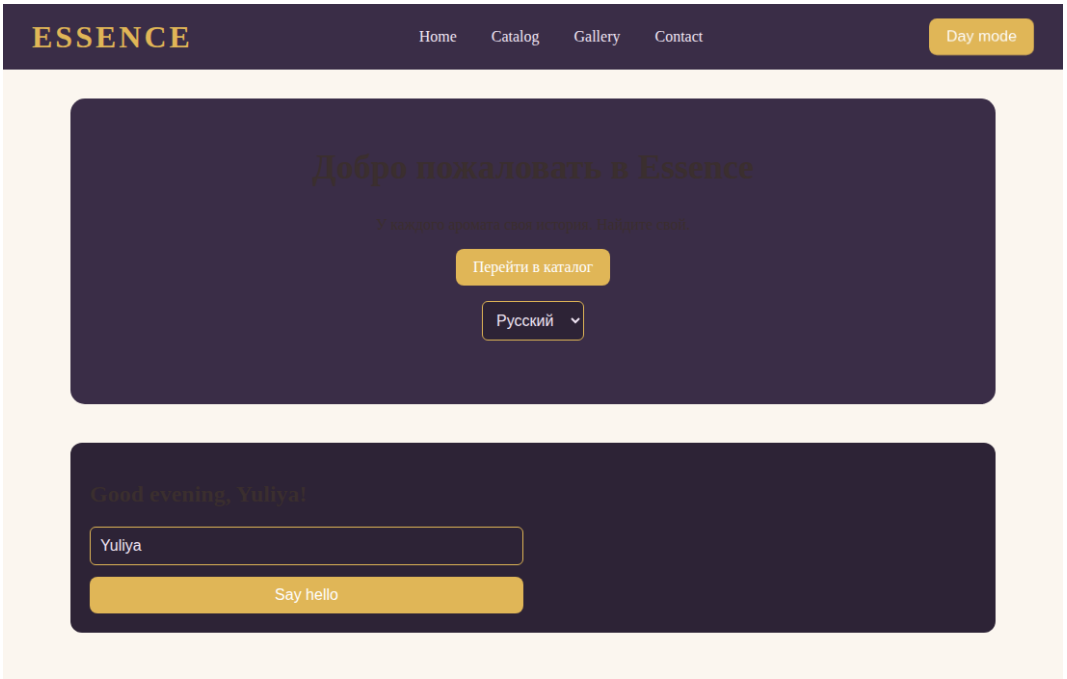
```js
// theme.js - Day / Night switch (all pages)
const themeBtn = document.getElementById('theme-btn');

// set the button text depending on the current theme
function updateButtonText() {
  if (document.body.classList.contains('night')) {
    themeBtn.textContent = 'Day mode';
  } else {
    themeBtn.textContent = 'Night mode';
  }
}

// remember the theme from the last visit
if (localStorage.getItem('theme') === 'night') {
  document.body.classList.add('night');
}
updateButtonText();

themeBtn.addEventListener('click', function () {
  document.body.classList.toggle('night');
  localStorage.setItem('theme', document.body.classList.contains('night') ? 'night' : 'day');
  updateButtonText();
});
```

Result:



Task 3.2. Greeting by time of day + name input (js/greeting.js)

Requirement: 1. Manipulating Attributes (greeting + input) / 2. Switch Statements
Author: Manyakina Yuliya

Step-by-step description:
1) getGreeting() reads the hour with new Date().getHours() and chooses morning / afternoon / evening.
2) A switch statement returns the correct greeting text.
3) On form submit, event.preventDefault() stops page reload and textContent updates the greeting with the user name.

Code:

```
// greeting.js - greeting by time of day + name from the input (index page)
const greeting = document.getElementById('greeting');
const nameForm = document.getElementById('name-form');
const nameInput = document.getElementById('name-input');

// returns a greeting text depending on the hour
function getGreeting() {
  const hour = new Date().getHours();
  let part;
  if (hour < 12) {
    part = 'morning';
  } else if (hour < 18) {
    part = 'afternoon';
  } else {
    part = 'evening';
  }

  switch (part) {
    case 'morning':
      return 'Good morning';
    case 'afternoon':
      return 'Good afternoon';
    default:
      return 'Good evening';
  }
}

greeting.textContent = getGreeting() + '!';

nameForm.addEventListener('submit', function (event) {
  event.preventDefault();                // do not reload the page
  const name = nameInput.value.trim();
  if (name === '') {
    greeting.textContent = getGreeting() + '!';
  } else {
    greeting.textContent = getGreeting() + ', ' + name + '!';
  }
});
```

Result:

English

Good evening, Yuliya!

Yuliya

Say hello

Store time

Show current time

9:24:25 PM

Task 3.3. Current time button (js/clock.js)

Requirement: 2. Event Handling - Event Listeners on Buttons
Author: Manyakina Yuliya

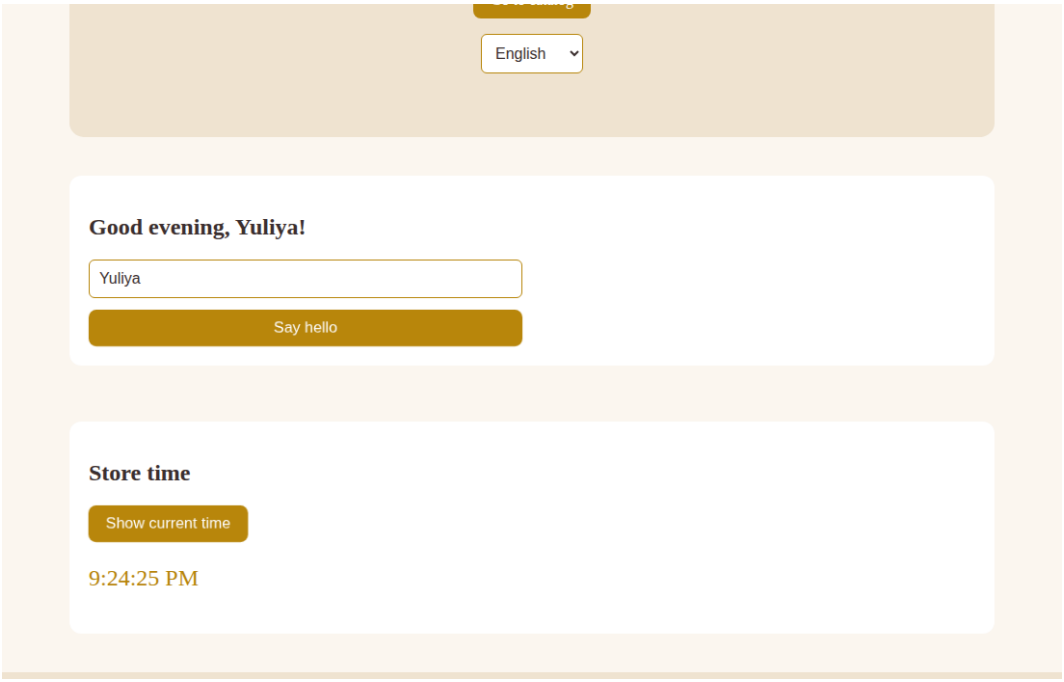
Step-by-step description:
1) Select the button and the output paragraph.
2) addEventListener("click") runs a function.
3) The function writes new Date().toLocaleTimeString() into the paragraph.

Code:

```
// clock.js - button that shows the current time (index page)
const timeBtn = document.getElementById('time-btn');
const timeOutput = document.getElementById('time-output');

timeBtn.addEventListener('click', function () {
  timeOutput.textContent = new Date().toLocaleTimeString();
});
```

Result:



Task 3.4. Keyboard navigation in the menu (js/nav.js)

Requirement: 2. Event Handling - Keyboard Events
Author: Manyakina Yuliya

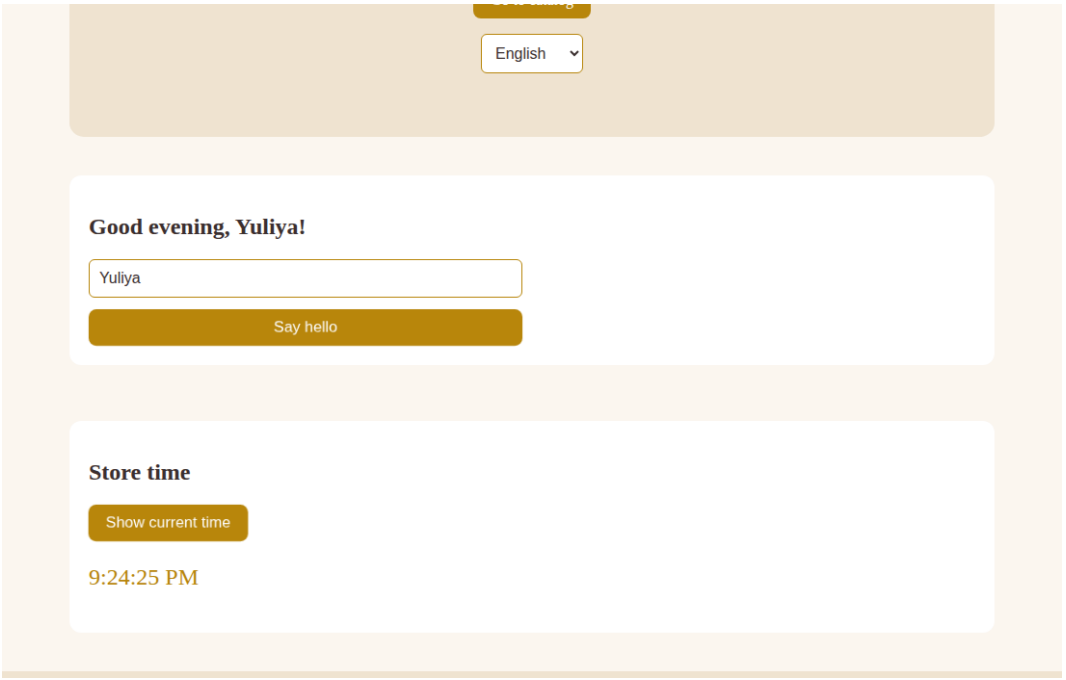
Step-by-step description:
1) querySelectorAll collects all menu links.
2) A keydown listener checks event.key (ArrowRight / ArrowLeft).
3) The index changes by 1 (with % so it loops) and focus() moves the focus to the link.

Code:

```
// nav.js - keyboard navigation in the menu with arrow keys (all pages)
const menuLinks = document.querySelectorAll('#menu a');
let currentIndex = 0;

document.addEventListener('keydown', function (event) {
  if (event.key === 'ArrowRight') {
    currentIndex = (currentIndex + 1) % menuLinks.length;   // next link, after the last go to the first
    menuLinks[currentIndex].focus();
  }
  if (event.key === 'ArrowLeft') {
    currentIndex = (currentIndex - 1 + menuLinks.length) % menuLinks.length; // previous link
    menuLinks[currentIndex].focus();
  }
});
```

Result:



Task 3.5. Language selector (EN / RU / KZ) (js/language.js)

Requirement: 2. Switch Statements - language selector
Author: Manyakina Yuliya

Step-by-step description:
1) All texts are stored in the object "translations".
2) On "change" of the select, a switch chooses the language object.
3) forEach goes through all elements with data-key and sets textContent from the object.

Code:

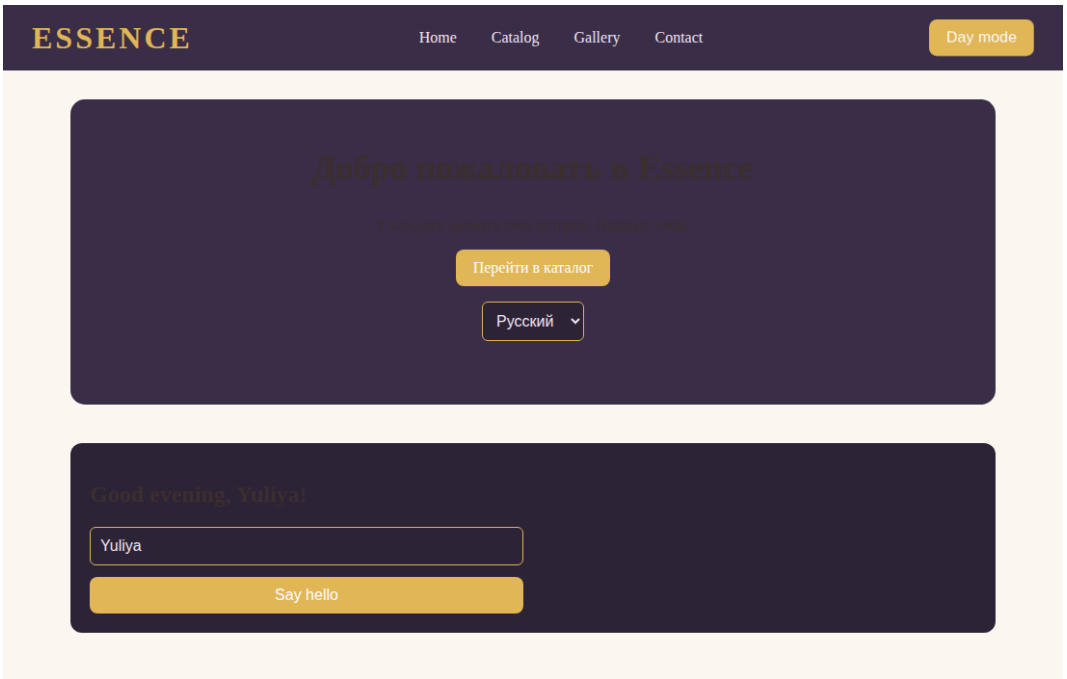
```js
// language.js - language selector English / Russian / Kazakh (index page)
// object with all texts, the keys are the same as data-key in HTML
const translations = {
  en: { title: 'Welcome to Essence', text: 'Every scent tells a story. Find yours.', btn: 'Go to catalog' },
  ru: { title: 'Добро пожаловать в Essence', text: 'У каждого аромата своя история. Найдите свой.', btn: 'Перейти в каталог' },
  kk: { title: 'Essence дүкеніне қош келдіңіз', text: 'Әр иістің өз тарихы бар. Өзіңіздікін табыңыз.', btn: 'Каталогқа өту' }
};

const langSelect = document.getElementById('lang-select');

langSelect.addEventListener('change', function () {
  let current;
  switch (langSelect.value) {
    case 'ru':
      current = translations.ru;
      break;
    case 'kk':
      current = translations.kk;
      break;
    default:
      current = translations.en;
  }

  // go through all elements with data-key and change their text
  document.querySelectorAll('[data-key]').forEach(function (element) {
    element.textContent = current[element.dataset.key];
  });
});
```

Result:



Task 3.6. Sound effects (js/sound.js)

Requirement: 3. Play Sounds
Author: Bazarbayev Bakdaulet

Step-by-step description:
1) An object "sounds" keeps Audio objects for click, star and success.
2) playSound(name) pauses the sound and sets currentTime = 0, so sounds do not overlap.
3) One click listener on document plays the click sound for every button.

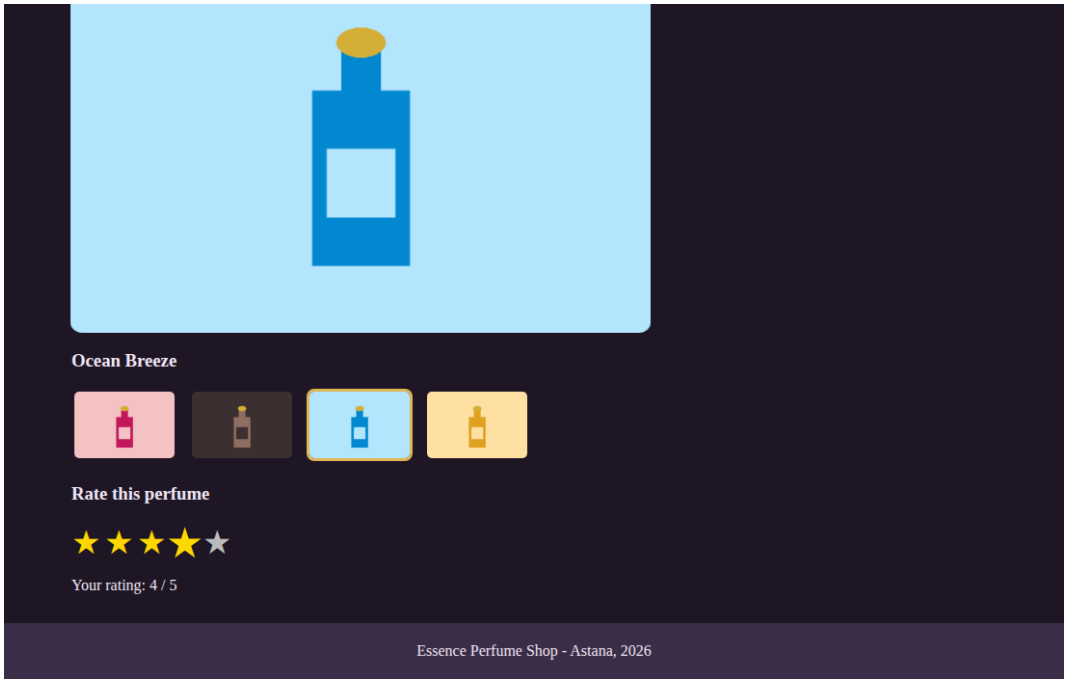
Code:

```
// sound.js - sound effects (all pages)
const sounds = {
  click: new Audio('sounds/click.wav'),
  star: new Audio('sounds/star.wav'),
  success: new Audio('sounds/success.wav')
};

function playSound(name) {
  const sound = sounds[name];
  sound.pause();
  sound.currentTime = 0;   // start from the beginning, so sounds do not overlap
  sound.play().catch(function () {});  // browser can block sound before the first click
}

// every button on the site makes a click sound
document.addEventListener('click', function (event) {
  if (event.target.tagName === 'BUTTON') {
    playSound('click');
  }
});
```

Result:



Task 3.7. Perfume data (objects and arrays) (js/data.js)

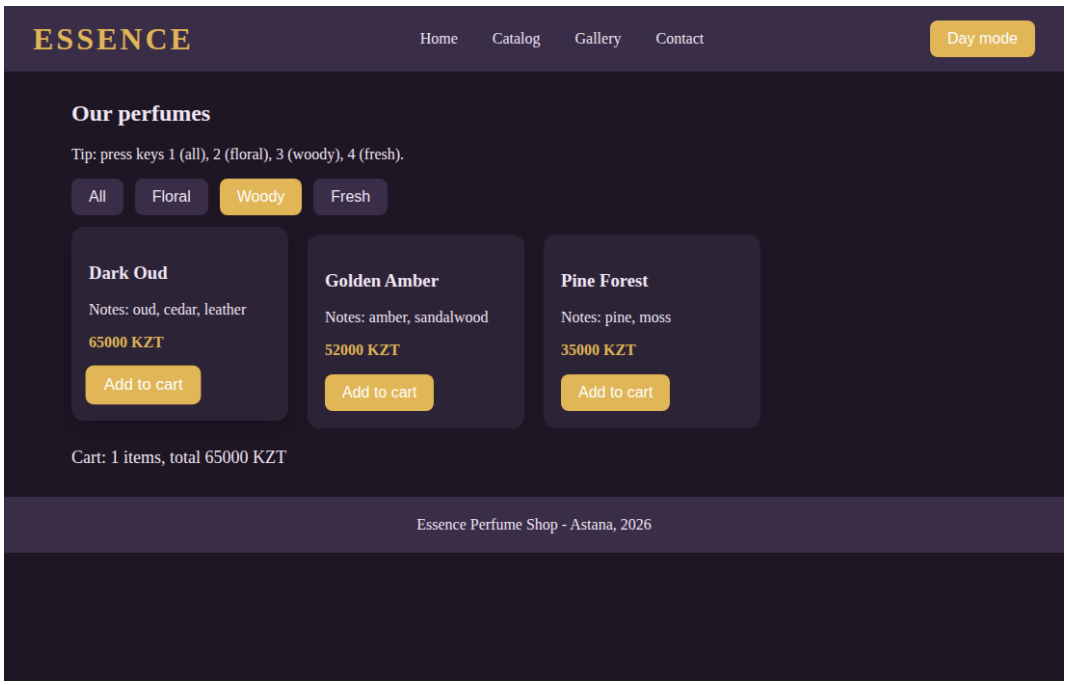
Requirement: 3. Objects and Methods / Arrays
Author: Bazarbayev Bakdaulet

Step-by-step description:
1) Every perfume is an object with id, name, category, price and notes.
2) All objects are stored in the array "perfumes".

Code:

```
// data.js - array of objects with our perfumes (catalog page)
const perfumes = [
  { id: 1, name: 'Rose Dream',    category: 'floral', price: 42000, notes: 'rose, peony, musk' },
  { id: 2, name: 'Jasmine Night', category: 'floral', price: 38000, notes: 'jasmine, vanilla' },
  { id: 3, name: 'Dark Oud',      category: 'woody',  price: 65000, notes: 'oud, cedar, leather' },
  { id: 4, name: 'Golden Amber',  category: 'woody',  price: 52000, notes: 'amber, sandalwood' },
  { id: 5, name: 'Ocean Breeze',  category: 'fresh',  price: 30000, notes: 'sea salt, lemon' },
  { id: 6, name: 'Green Tea',     category: 'fresh',  price: 28000, notes: 'green tea, mint' },
  { id: 7, name: 'Velvet Iris',   category: 'floral', price: 47000, notes: 'iris, violet' },
  { id: 8, name: 'Pine Forest',   category: 'woody',  price: 35000, notes: 'pine, moss' }
];
```

Result:



Task 3.8. Catalog: filter, cart, higher-order functions (js/catalog.js)

Requirement: 2. Switch Statements (filter) / 3. Higher-Order Functions, Arrays and Loops
Author: Bazarbayev Bakdaulet

Step-by-step description:
1) makeCard() builds HTML for one perfume; map() + join() renders the whole list.
2) showPerfumes(category) uses a switch with filter() to choose perfumes of one category.
3) Event delegation: one click listener on the grid handles all "Add to cart" buttons; find() gets the perfume by id.
4) reduce() calculates the cart total. Keys 1-4 (keydown + switch) also change the filter. Cards use CSS fadeIn animation.

Code:

```
// catalog.js - filter perfumes by category and a simple cart (catalog page)
const grid = document.getElementById('perfume-grid');
const filterButtons = document.querySelectorAll('.filter-btn');
const cartInfo = document.getElementById('cart-info');
let cart = [];   // array of perfumes that user added

// makes HTML for one perfume card
function makeCard(perfume) {
  return '<div class="card">' +
    '<h3>' + perfume.name + '</h3>' +
    '<p>Notes: ' + perfume.notes + '</p>' +
    '<p class="price">' + perfume.price + ' KZT</p>' +
    '<button class="add-btn" data-id="' + perfume.id + '">Add to cart</button>' +
    '</div>';
}

// shows perfumes of the chosen category
function showPerfumes(category) {
  let list;
  switch (category) {
    case 'floral':
      list = perfumes.filter(function (p) { return p.category === 'floral'; });
      break;
    case 'woody':
      list = perfumes.filter(function (p) { return p.category === 'woody'; });
      break;
    case 'fresh':
      list = perfumes.filter(function (p) { return p.category === 'fresh'; });
      break;
    default:
      list = perfumes;
  }
  grid.innerHTML = list.map(makeCard).join('');

  // highlight the active filter button
  filterButtons.forEach(function (btn) {
    btn.classList.toggle('active', btn.dataset.category === category);
  });
}

// updates the text under the grid
function updateCart() {
  const total = cart.reduce(function (sum, p) { return sum + p.price; }, 0);
  cartInfo.textContent = 'Cart: ' + cart.length + ' items, total ' + total + ' KZT';
}

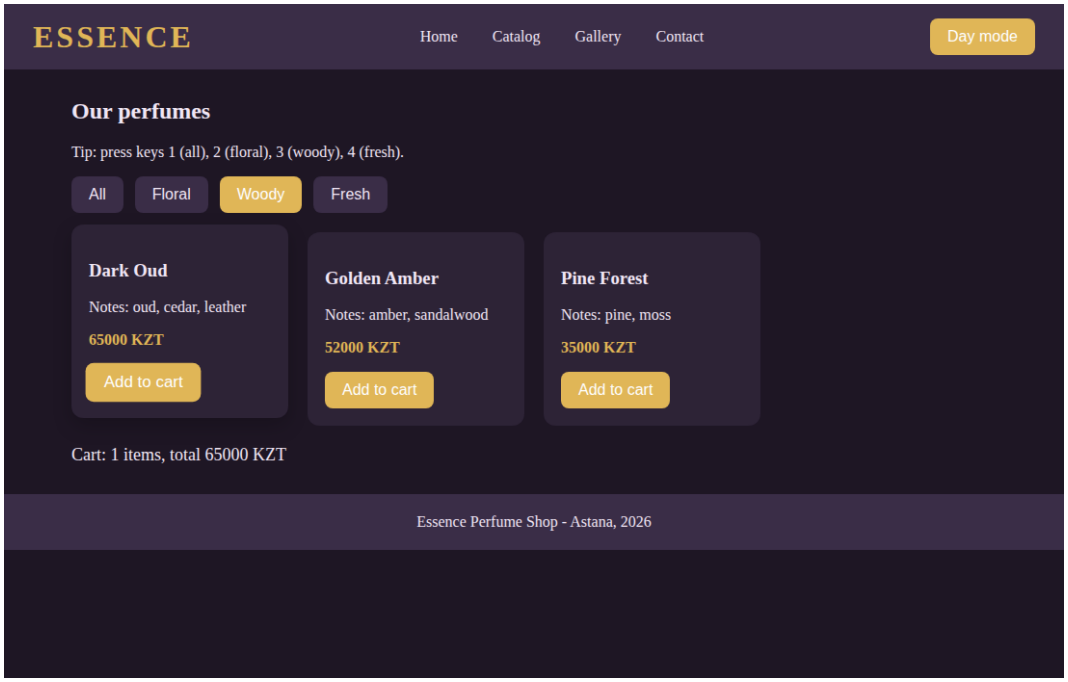
filterButtons.forEach(function (btn) {
  btn.addEventListener('click', function () {
    showPerfumes(btn.dataset.category);
  });
});

// one listener for all "Add to cart" buttons
grid.addEventListener('click', function (event) {
  if (event.target.classList.contains('add-btn')) {
    const id = Number(event.target.dataset.id);
    const perfume = perfumes.find(function (p) { return p.id === id; });
    cart.push(perfume);
    updateCart();
  }
});

// keyboard shortcuts 1-4
document.addEventListener('keydown', function (event) {
  switch (event.key) {
    case '1': showPerfumes('all'); break;
    case '2': showPerfumes('floral'); break;
    case '3': showPerfumes('woody'); break;
    case '4': showPerfumes('fresh'); break;
  }
});

showPerfumes('all');
```

Result:


ESSENCE
Home
Catalog
Gallery
Contact
Day mode
Our perfumes
Tip: press keys 1 (all), 2 (floral), 3 (woody), 4 (fresh).
All
Floral
Woody
Fresh
Dark Oud
Notes: oud, cedar, leather
65000 KZT
Add to cart
Golden Amber
Notes: amber, sandalwood
52000 KZT
Add to cart
Pine Forest
Notes: pine, moss
35000 KZT
Add to cart
Cart: 1 items, total 65000 KZT
Essence Perfume Shop - Astana, 2026

Task 3.9. Image gallery with thumbnails (js/gallery.js)

Requirement: 1. Manipulating Attributes - image gallery / 3. Animations
Author: Bazarbayev Bakdaulet

Step-by-step description:
1) querySelectorAll selects all thumbnails.
2) On click the src of the big image is replaced with the src of the thumbnail.
3) The class "active" is moved to the clicked thumbnail, and the class "pop" gives a short zoom animation (removed by setTimeout).

Code:

```
// gallery.js - thumbnails change the big image (gallery page)
const bigImage = document.getElementById('big-image');
const imageTitle = document.getElementById('image-title');
const thumbs = document.querySelectorAll('#thumbs img');

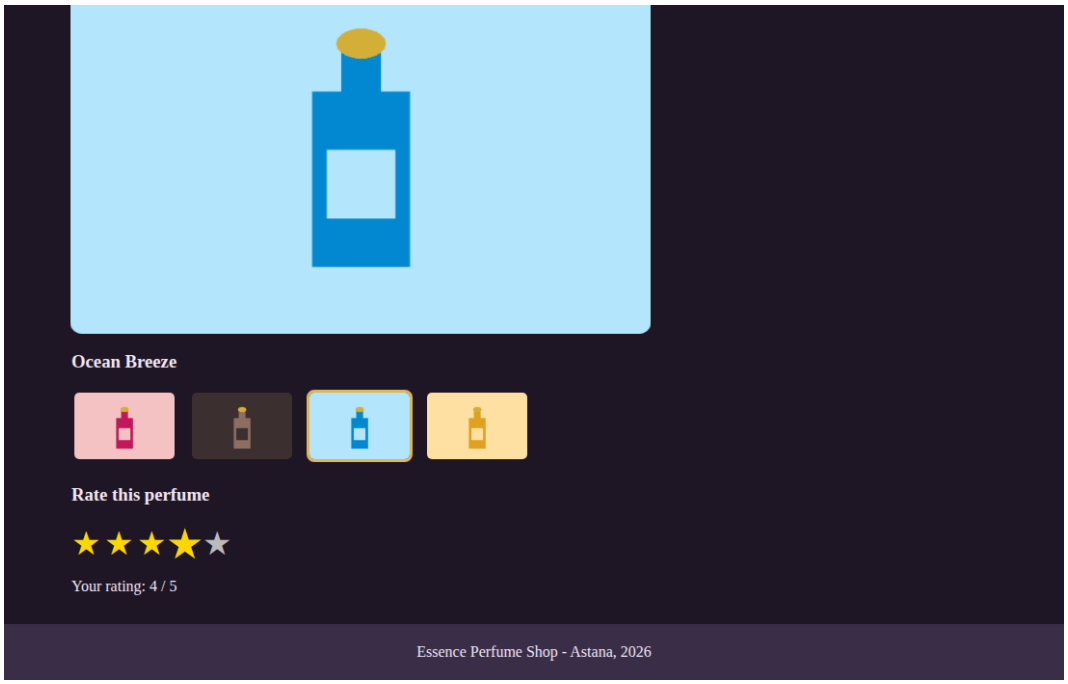
thumbs.forEach(function (thumb) {
  thumb.addEventListener('click', function () {
    bigImage.src = thumb.src;            // change the src attribute
    imageTitle.textContent = thumb.alt;  // change the title

    // move the "active" class to the clicked thumbnail
    thumbs.forEach(function (t) { t.classList.remove('active'); });
    thumb.classList.add('active');

    // small zoom animation
    bigImage.classList.add('pop');
    setTimeout(function () { bigImage.classList.remove('pop'); }, 300);

    showSavedRating();   // function from rating.js
  });
});
```

Result:



Task 3.10. Star rating (js/rating.js)

Requirement: 1. DOM Manipulation - rating stars
Author: Bazarbayev Bakdaulet

Step-by-step description:
1) querySelectorAll(".star") selects all stars.
2) On click the rating is saved into the object "ratings" (perfume name -> value).
3) paintStars() uses forEach + classList.toggle to color the first N stars gold.
4) Star sound is played with playSound("star").

Code:

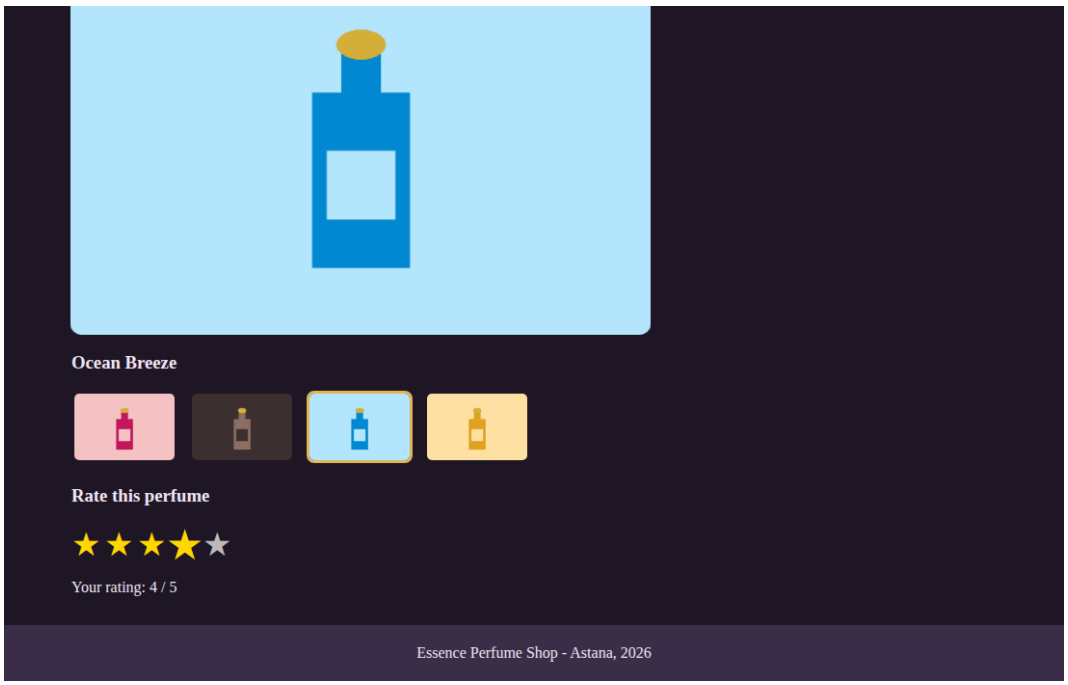
```
// rating.js - star rating for every perfume (gallery page)
const stars = document.querySelectorAll('.star');
const ratingText = document.getElementById('rating-text');
const ratings = {};   // object: perfume name -> rating, for example { 'Rose Dream': 4 }

// colors the first "value" stars
function paintStars(value) {
  stars.forEach(function (star, index) {
    star.classList.toggle('active', index < value);
  });
}

// shows the rating of the current perfume (called also from gallery.js)
function showSavedRating() {
  const name = document.getElementById('image-title').textContent;
  const value = ratings[name] || 0;
  paintStars(value);
  ratingText.textContent = value === 0 ? 'No rating yet' : 'Your rating: ' + value + ' / 5';
}

stars.forEach(function (star) {
  star.addEventListener('click', function () {
    const name = document.getElementById('image-title').textContent;
    ratings[name] = Number(star.dataset.value);
    showSavedRating();
    playSound('star');
  });
});
```

Result:



Task 3.11. Contact form with fetch and callbacks (js/contact.js)

Requirement: 2. Responding to Events with Callbacks / Reset button
Author: Bazarbayev Bakdaulet

Step-by-step description:
1) The submit event is stopped with preventDefault() and fields are checked.
2) sendMessage(data, onSuccess, onError) sends POST with fetch; the two callbacks show the result message.
3) The Clear button uses querySelectorAll("input, textarea").forEach to empty all fields.

Code:

```
// contact.js - send the form without reloading the page (contact page)
const form = document.getElementById('contact-form');
const message = document.getElementById('form-message');
const resetBtn = document.getElementById('reset-btn');

// sends data to the server; onSuccess and onError are callback functions
function sendMessage(data, onSuccess, onError) {
  fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data)
  })
    .then(function (response) { return response.json(); })
    .then(onSuccess)
    .catch(onError);
}

function clearInputs() {
  document.querySelectorAll('input, textarea').forEach(function (field) {
    field.value = '';
  });
}

function showSuccess(result) {
  message.textContent = 'Thank you! Your message was sent (id: ' + result.id + ').';
  message.className = 'success';
  clearInputs();
  playSound('success');
}

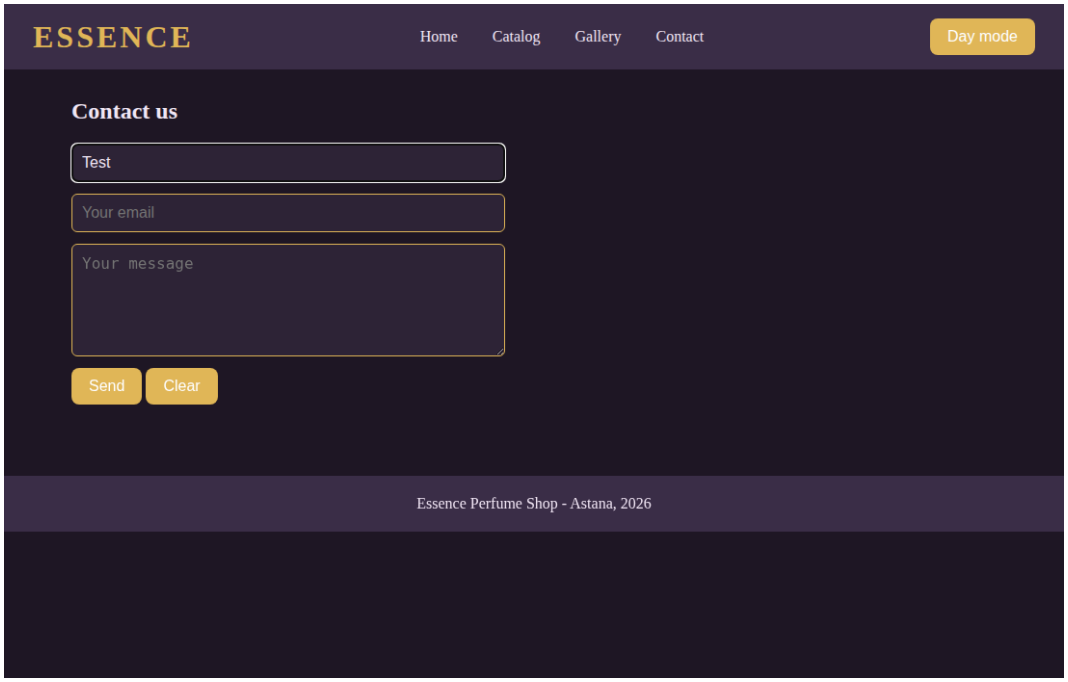
function showError() {
  message.textContent = 'Something went wrong. Please try again.';
  message.className = 'error';
}

form.addEventListener('submit', function (event) {
  event.preventDefault();
  const data = {
    name: document.getElementById('name').value.trim(),
    email: document.getElementById('email').value.trim(),
    message: document.getElementById('message').value.trim()
  };

  if (data.name === '' || data.email === '' || data.message === '') {
    message.textContent = 'Please fill in all fields.';
    message.className = 'error';
    return;
  }
  message.textContent = 'Sending...';
  message.className = '';
  sendMessage(data, showSuccess, showError);
});

resetBtn.addEventListener('click', clearInputs);
```

Result:


ESSENCE
Home
Catalog
Gallery
Contact
Day mode
Contact us
Test
Your email
Your message
Send
Clear
Essence Perfume Shop - Astana, 2026

4. Structure and clean code

HTML files contain only structure, css/style.css contains all styles (colors in CSS variables), and each script is in its own file in js/. There is no inline CSS or inline JavaScript. Variables have meaningful names and every script starts with a comment that explains what it does.

5. Deployment (Part 2)

1) Upload the project folder to the shared GitHub repository (git add, git commit, git push).
2) In the repository open Settings - Pages and choose the main branch, root folder.
3) Wait one minute and open the generated link.
URL: [PASTE URL HERE]

6. Reflection

The hardest parts were the fetch request with callbacks and the rating that is saved separately for every perfume. We solved the first one by splitting the code into small functions (sendMessage, showSuccess, showError), and the second one by saving ratings in an object where the key is the perfume name. We also learned that sounds must be restarted with currentTime = 0, otherwise they overlap.